

Table of Contents

Task 1: The Long-Running Process	3
Task 2: Process Identity (PID and PPID)	4
Task 3: Exit Codes and OS Feedback	5
Task 4: Standard I/O Streams	6
Task 5: Conditional Execution and Termination	7

Table of Figures

Figure 1: Error I got first time in task1_alive.c	3
Figure 2: Task 1 running in background, checked with ps	3
Figure 3: My task2_identity.c code	4
Figure 4: Task 2 output showing PID and PPID	4
Figure 5: My task3_exit.c code	5
Figure 6: Task 3 output, exit code checked with echo \$?	5
Figure 7: My task4_input.c code	6
Figure 8: Task 4 output asking my name	6
Figure 9: My task5_control.c code	7
Figure 10: Task 5 output with yes or no answer	7

Task 1: The Long-Running Process

Big programs can run for a long time, so we need a way to check them while they are still running. So here, I write a program that run for 30 second. It print starting message, wait using sleep, then print finish message. I run it in the background with & so I can check it from another window while it is still running.

```
kshitz@kshitz-VirtualBox:~/linux_lab$ ls
hello hello.c hello.i hello.o hello.s task1_alive.c
kshitz@kshitz-VirtualBox:~/linux_lab$ gcc task1_alive.c -o task1
task1_alive.c: In function 'main':
task1_alive.c:9:1: error: expected statement before ')' token
   9 | )
     | ^
task1_alive.c:13:1: error: expected declaration or statement at end of input
   13 | }
      | ^
kshitz@kshitz-VirtualBox:~/linux_lab$
```

Figure 1: Error I got first time in task1_alive.c

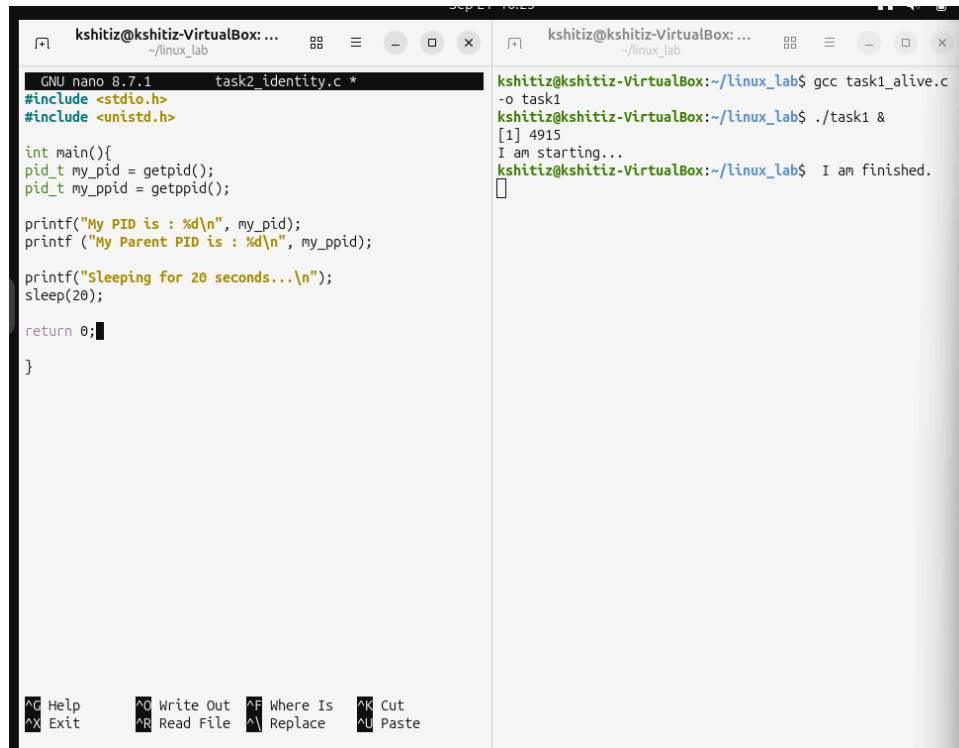
```
kshitz@kshitz-VirtualBox:~/linux_lab$ ls
hello hello.c hello.i hello.o hello.s task1_alive.c
kshitz@kshitz-VirtualBox:~/linux_lab$ gcc task1_alive.c -o task1
task1_alive.c: In function 'main':
task1_alive.c:9:1: error: expected statement before ')' token
   9 | )
     | ^
task1_alive.c:13:1: error: expected declaration or statement at end of input
   13 | }
      | ^
kshitz@kshitz-VirtualBox:~/linux_lab$ nano task1_alive.c
kshitz@kshitz-VirtualBox:~/linux_lab$ gcc task1_alive.c -o task1
kshitz@kshitz-VirtualBox:~/linux_lab$ ls
hello hello.c hello.i hello.o hello.s task1 task1_alive.c
kshitz@kshitz-VirtualBox:~/linux_lab$ ./task1
I am starting...
I am finished.
kshitz@kshitz-VirtualBox:~/linux_lab$ ps aux | grep task
1 kshitz 4763 0.0 0.0 18000 2644 pts/0 S+ 10:1
5 0:00 grep --color=auto task1
kshitz@kshitz-VirtualBox:~/linux_lab$
```

Figure 2: Task 1 running in background, checked with ps

Task 2: Process Identity (PID and PPID)

Note: Every process in Linux have own number call PID. The process that start it have number call PPID, mostly it is the terminal.

I write a program using getpid() and getppid() to print my own PID and PPID. Then I sleep 20 second so I can check it from other terminal.



```
GNU nano 8.7.1 task2_identity.c *
#include <stdio.h>
#include <unistd.h>

int main(){
    pid_t my_pid = getpid();
    pid_t my_ppid = getppid();

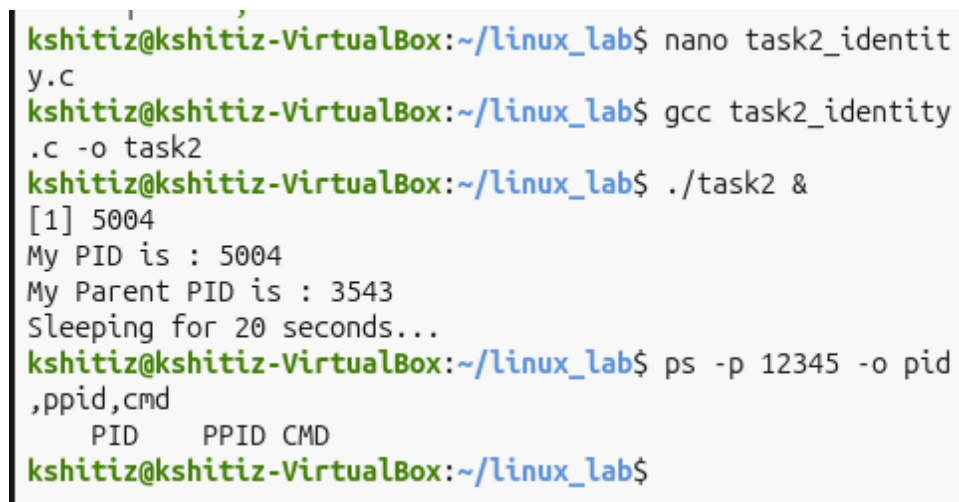
    printf("My PID is : %d\n", my_pid);
    printf("My Parent PID is : %d\n", my_ppid);

    printf("Sleeping for 20 seconds...\n");
    sleep(20);

    return 0;
}

kshitz@kshitz-VirtualBox: ~/linux_lab$ gcc task1_alive.c
-o task1
kshitz@kshitz-VirtualBox:~/linux_lab$ ./task1 &
[1] 4915
I am starting...
kshitz@kshitz-VirtualBox:~/linux_lab$ I am finished.
```

Figure 3: My task2_identity.c code

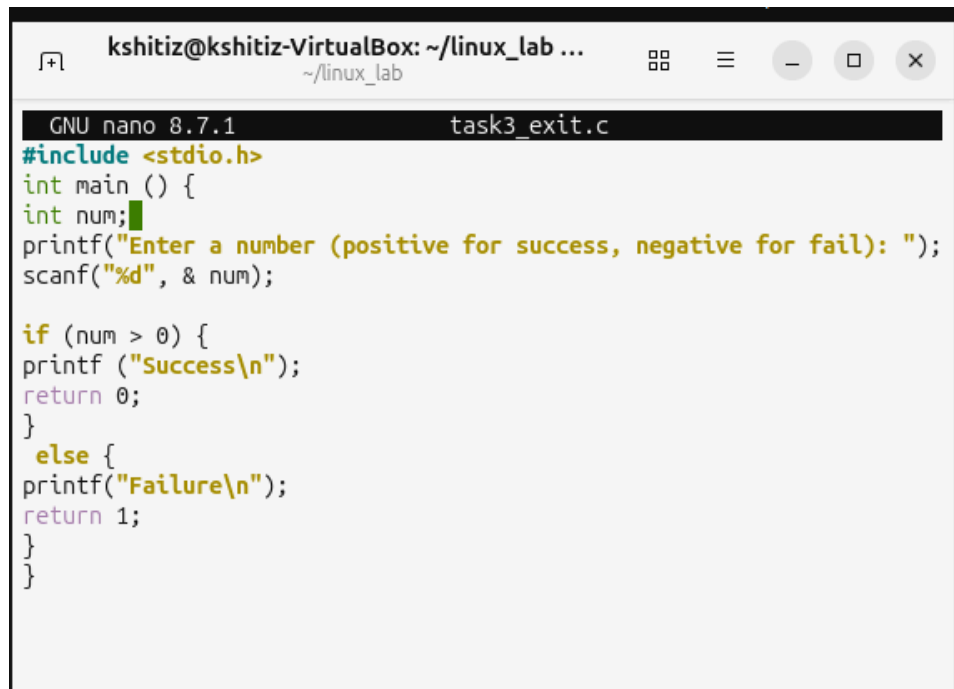


```
kshitz@kshitz-VirtualBox:~/linux_lab$ nano task2_identity.c
kshitz@kshitz-VirtualBox:~/linux_lab$ gcc task2_identity.c -o task2
kshitz@kshitz-VirtualBox:~/linux_lab$ ./task2 &
[1] 5004
My PID is : 5004
My Parent PID is : 3543
Sleeping for 20 seconds...
kshitz@kshitz-VirtualBox:~/linux_lab$ ps -p 12345 -o pid,ppid,cmd
PID      PPID CMD
kshitz@kshitz-VirtualBox:~/linux_lab$
```

Figure 4: Task 2 output showing PID and PPID

Task 3: Exit Codes and OS Feedback

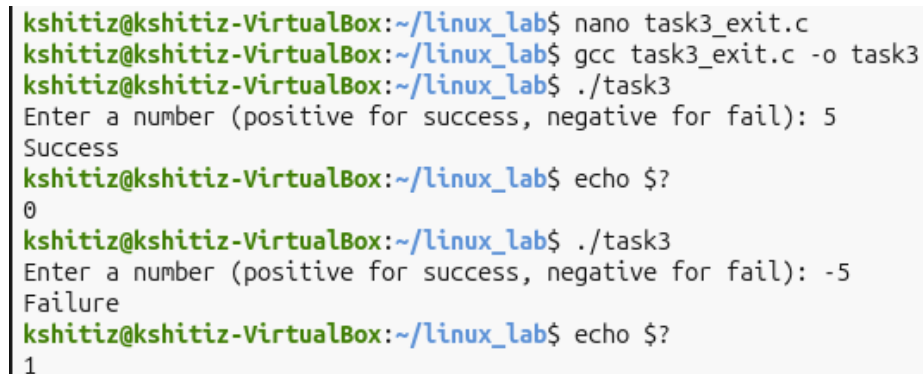
When program finish, it give number back to OS. 0 mean success and other number like 1 mean fail. Shell keep this number in \$? I write program that ask for number. If number is positive it print Success and return 0. If not it print Failure and return 1.

A screenshot of a terminal window titled 'kshitiz@kshitiz-VirtualBox: ~/linux_lab ...'. The window shows the nano 8.7.1 editor editing a file named 'task3_exit.c'. The code is as follows:

```
GNU nano 8.7.1 task3_exit.c
#include <stdio.h>
int main () {
int num;
printf("Enter a number (positive for success, negative for fail): ");
scanf("%d", & num);

if (num > 0) {
printf ("Success\n");
return 0;
}
else {
printf("Failure\n");
return 1;
}
}
```

Figure 5: My task3_exit.c code

A screenshot of a terminal window showing the compilation and execution of the program. The commands and their outputs are:

```
kshitiz@kshitiz-VirtualBox:~/linux_lab$ nano task3_exit.c
kshitiz@kshitiz-VirtualBox:~/linux_lab$ gcc task3_exit.c -o task3
kshitiz@kshitiz-VirtualBox:~/linux_lab$ ./task3
Enter a number (positive for success, negative for fail): 5
Success
kshitiz@kshitiz-VirtualBox:~/linux_lab$ echo $?
0
kshitiz@kshitiz-VirtualBox:~/linux_lab$ ./task3
Enter a number (positive for success, negative for fail): -5
Failure
kshitiz@kshitiz-VirtualBox:~/linux_lab$ echo $?
1
```

Figure 6: Task 3 output. exit code checked with echo \$?

I run program two time. First time I type 5, it print Success and echo \$? show 0. Second time I type -5, it print Failure and echo \$? show 1. This show return value inside program is what shell get as exit code.

Task 4: Standard I/O Streams

OS give every program way to read input (stdin) and give output (stdout). scanf read from stdin and printf write to stdout. I write program that ask my name using scanf and then print hello message with my name using printf.

A screenshot of a terminal window. The title bar shows 'kshitiz@kshitiz-VirtualBox: ~/linux'. The terminal content shows the nano editor interface with the file 'task4' open. The code is as follows:

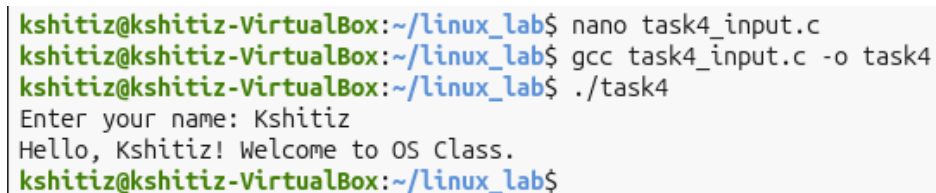
```
GNU nano 8.7.1 task4
#include <stdio.h>

int main () {
char name [50];

printf("Enter your name: ");
scanf("%s", name);

printf ("Hello, %s! Welcome to OS Class.\n", name);
return 0;
}
```

Figure 7: My task4_input.c code

A screenshot of a terminal window showing the execution of the program. The commands and output are:

```
kshitiz@kshitiz-VirtualBox:~/linux_lab$ nano task4_input.c
kshitiz@kshitiz-VirtualBox:~/linux_lab$ gcc task4_input.c -o task4
kshitiz@kshitiz-VirtualBox:~/linux_lab$ ./task4
Enter your name: Kshitiz
Hello, Kshitiz! Welcome to OS Class.
kshitiz@kshitiz-VirtualBox:~/linux_lab$
```

Figure 8: Task 4 output asking my name

I run the program and type my name Kshitiz. It print "Hello, Kshitiz! Welcome to OS Class." This show how scanf and printf use the input/output stream from OS.

Task 5: Conditional Execution and Termination

Program can choose different path base on input. OS or other program can look at final exit code to know continue or stop.

I write a program that show its PID and ask yes or no question. If I say 1 (yes) it continue and return 0. If I say 0 (no) it stop and return 1.

```
GNU nano 2.7.1 task5.c
#include <stdio.h>
#include <unistd.h>

int main() {
    int choice;

    printf("Current PID: %d\n", getpid());

    printf("Do you want to continue? (1 for Yes, 0 for No): ");
    scanf("%d", &choice);

    if (choice == 1) {
        printf("Continuing...\n");
        sleep(5);
        return 0;
    }
    else{
        printf("Existing.....\n");
        return 1;
    }
}
```

Figure 9: My task5_control.c code

```
kshitiz@kshitiz-VirtualBox:~/linux_lab$ nano task5_control.c
kshitiz@kshitiz-VirtualBox:~/linux_lab$ gcc task5_control -o task5
/usr/bin/x86_64-linux-gnu-ld.bfd: cannot find task5_control: No such file or directory
collect2: error: ld returned 1 exit status
kshitiz@kshitiz-VirtualBox:~/linux_lab$ gcc task5_control.c -o task5
kshitiz@kshitiz-VirtualBox:~/linux_lab$ ./task5
Current PID: 9730
Do you want to continue? (1 for Yes, 0 for No): y
Existing.....
kshitiz@kshitiz-VirtualBox:~/linux_lab$ ./task5
Current PID: 9737
Do you want to continue? (1 for Yes, 0 for No): 0
Existing.....
kshitiz@kshitiz-VirtualBox:~/linux_lab$
```

Figure 10: Task 5 output with yes or no answer

First time I compile wrong, I forget .c at end of file name, so I get error. After I fix, program compile good. I run it two time, but both time I answer in way that go to else part, so I see "Existing....." both time. I should try answer 1 also to see the "Continuing" part.